



FACULTY OF ACCOUNTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III

TEST 3 (Intersystem communication) Memo

DATE: 27 MAY 2025

TIME: 45 minutes

EXAMINER: BOAMAH-ABU, CHARLES (PhD)

TOTAL MARKS: 35

NO. OF PAGES: 6 (Including cover page)

INSTRUCTIONS/REQUIREMENTS

1. *This examination is closed book*
2. *Answer all questions*
3. *Where stated "Briefly answer", please give a short answer*
4. *Where stated "Write the missing parts" of code, please write only the missing parts*
5. *Ensure your handwriting is readable*
6. **NOTE: Java is case sensitive**

Question 1

10 marks

In each of the following, state whether the statement is **true** or **false**. Write **T** (for true) or **F** (for false)

1. Distributed architectures are design patterns.
2. In distributed applications, strong coupling between the applications is preferable.
3. In distributed applications, strong cohesion within applications is preferable.
4. In the broker architecture, the client and server communicate directly.
5. Microservices consist of large-scale distributed applications.
6. Sockets allow low-level network programming.
7. In socket programming, the client listens for a request.
8. In socket programming, a server has an output stream only.
9. RMI builds hides low-level communication.
10. RMI is useful in Integrative programming because it is language independent.

Answer: 1. T 2. F 3. T 4. F 5. F 6. T 7. F 8. F 9. T 10. F

Question 2 (Socket programming)

12 marks

In a socket application, the **client application** writes two numbers into a stream; the **server application** reads the two numbers, finds the maximum of the two number and writes this to the stream. The communication port number is **2025** and the server name is **localhost**. The following are the classes for the client and the server applications.

 Client  Client()  client(double, double) : void	 Server  Server()  server() : void
 ClientTester  main(String[]) : void	 ServerTester  main(String[]) : void

The class **Client** contains a **constructor** and a method **client** (which takes two numbers as parameters). **client** performs the following:

- a. Declares exceptions
- b. Creates the client-side socket
- c. Creates an appropriate stream to write two numbers (the parameters x and y) into the socket
- d. Creates an appropriate stream to read a number from the socket and write this on the screen
- e. Closes the streams and socket

The class **ClientTester** contains a method **main** which creates an instance of **Client** and calls the method **client**.

Use the information above to answer the following questions.

1. Below is the class **Client**. Complete the method **client** as follows (**write only the missing parts**):

1.1 Complete the method header of client to declare exceptions. (1)

1.2 Complete the line of code to create the client socket. (2)

1.3 Complete the lines of code which writes the parameters of **client** into the socket. (4)

1.4 Complete the line of code which reads a number from the socket and displays on the screen. (2)

```
2. package clientserver;
import java.io.*;
import java.net.*;

public class Client {

    public Client() {

    }

    public void client(double x, double y) _____{
        //create a socket
        Socket s=_____;

        //create output stream
        OutputStream os=s.getOutputStream();
        DataOutputStream dos=new DataOutputStream(os);

        //write the parameters x and y
        _____;
        _____;

        //create input stream
        InputStream is=s.getInputStream();
        DataInputStream dis=new DataInputStream(is);

        //read the maximum number from socket and write to the screen
        System.out.println("The maximum=" + _____);

        //close streams and socket
        dis.close();
        dos.close();
        s.close();

    }

}
```

Answer

```
...
...
public void client(double x, double y) throws Exception{
    //create a socket
    Socket s=new Socket("localhost", 2025); ✓✓

    //create output streams and write the parameters
    OutputStream os=s.getOutputStream();
    DataOutputStream dos=new DataOutputStream(os);

    // write the parameters
    dos.writeDouble(x); ✓✓
    dos.writeDouble(y); ✓✓

    //create input stream
    InputStream is=s.getInputStream();
    DataInputStream dis=new DataInputStream(is);
    // read a number (maximum) from the stream and write to the screen
    System.out.println("The maximum of is " + dis.readDouble());✓✓

    //close streams and socket
    dis.close();
    dos.close();
    s.close();

}
}
```

2. Below is the class **ClientTester**. Complete the method **main** as follows (write only the missing parts):

2.1 Complete the header of the method client to declare the exceptions from **client**. (1)

2.2 **c** is an instance of the class **Client**. Write the line of code to call the appropriate method to display on the screen, the maximum of **20** and **10**. (2)

```
package clientserver;

public class ClientTester {

    public static void main(String[] args)_____{
        Client c=new Client();
        _____;
    }

}
```

Answer

```
package clientserver;

public class ClientTester {

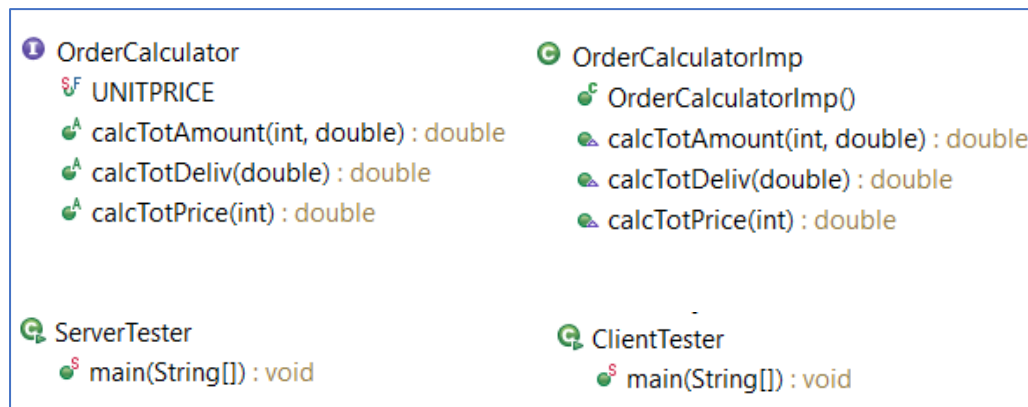
    public static void main(String[] args) throws Exception{
        Client c=new Client();
        c.client(20, 10); ✓✓
    }

}
```

Question 3 (RMI application)

13 marks

Below are the parts of an RMI application.



OrderCalculator is a remote interface. **OrderCalculatorImp** is the class that implements the remote interface. **ServerTester** is a class which creates and registers a remote object of **OrderCalculatorImp**. **ClientTester** calls the methods of the remote object.

1. Below is the code for an interface **OrderCalculator**. Convert to a remote interface. (4)

```
package rmi;
import java.rmi.*;

public interface OrderCalculator{
    final double UNITPRICE=10.0;
    double calcTotPrice(int qty);
    double calcTotDeliv(double distance);
    double calcTotAmount(int qty, double distance);
}
```

Answer

```
package rmi;

import java.rmi.*;

public interface OrderCalculator extends Remote✓ {
    final double UNITPRICE=10.0;
    double calcTotPrice(int qty) throws Exception✓;
    double calcTotDeliv(double distance) throws Exception✓;
    double calcTotAmount(int qty, double distance) throws Exception✓;
}
```

2. Below is a segment of the class **OrderCalculatorImp**. Complete the header. (3)

```
...
...
public class OrderCalculatorImp _____{
    ...
    ...
}
```

Answer

```
public class OrderCalculatorImp extends UnicastRemoteObject✓✓
implements OrderCalculator✓{ ...}
```

3. Below is the code for the class **ServerTester**. Complete the missing parts as follows:

3.1 Create a remote object (called **oc1**) of **OrderCalculatorImp**. (1)

3.2 Create a registry (called **r**) with a port number **2025**. (2)

3.3 Bind the remote object (**oc1**) to the registry (**r**) with a reference “**order**”. (3)

```
package rmi;
import java.rmi.registry.*;
public class ServerTester {

    public static void main(String[] args) throws Exception {
        //3.1 Create remote object
        _____

        //3.2 Create registry
        Registry r=_____

        //3.3 Bind remote object to registry
        _____
    }
}
```

Answer

```
package rmi;
import java.rmi.registry.*;

public class ServerTester {

    public static void main(String[] args) throws Exception {
        //3.1 Create remote object
        OrderCalculatorImp oc1=new OrderCalculatorImp();✓
        //3.2 Create registry
        Registry r=LocateRegistry.createRegistry(2025); ✓✓
        //3.3 Bind remote object to registry
        r.bind("order", oc1); ✓✓✓
    }
}
```